

COMP90038
Algorithms and Complexity

Cezary Kaliszyk
(thanks to Harald Søndergaard)

Lecture 20

Greedy Algorithms: Prim and Dijkstra

Dynamic Programming (Recap)

Knapsack problem

Interesting substructure relation

Warshall algorithm

Computing the transitive closure of a directed graph

Floyd's algorithm

Finding shortest distances in weighted directed graphs

Greedy Algorithms

Make decisions based on locally best choice

Only sometimes best global solution

Greedy Algorithms

Make decisions based on locally best choice

Only sometimes best global solution



Suppose we have coin denominations 25, 10, and 1, and we want to change 35 cents using the smallest number of coins.

Greedy Algorithms

Make decisions based on locally best choice

Only sometimes best global solution



Suppose we have coin denominations 25, 10, and 1, and we want to change 35 cents using the smallest number of coins.

In general we will want to use as many 25-cent pieces as we can, then do the same for 10-cent pieces, and so on, until we have reached 35 cents. (In this case we use 25+10 cents.)

This greedy strategy will work for 35 cents but not always. For which of these it will not work? (a) 50 (b) 30 (c) 5.

Greedy Algorithms

Locally best choices

Do not always yield globally best outcomes

Some well-known problems

- rely on the greedy approach
- correct (globally best) and fast

Hard problems

Greedy algorithms sometimes give acceptable approximation

Today

- Prim's algorithm for finding minimum spanning trees
- Dijkstra's algorithm for single-source shortest paths

Spanning Trees

Definition: Tree

connected graph with no cycles

Definition: Spanning tree

A spanning tree of a graph $\langle V, E \rangle$ is a tree $\langle V, E' \rangle$ with $E' \subseteq E$.

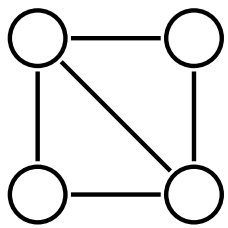
Spanning Trees

Definition: Tree

connected graph with no cycles

Definition: Spanning tree

A spanning tree of a graph $\langle V, E \rangle$ is a tree $\langle V, E' \rangle$ with $E' \subseteq E$.

The graph  has different spanning trees:

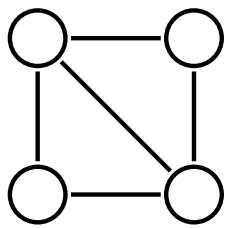
Spanning Trees

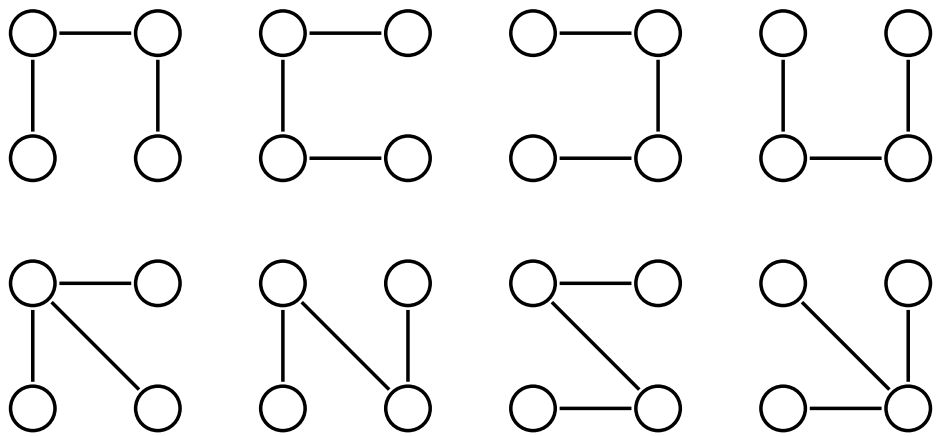
Definition: Tree

connected graph with no cycles

Definition: Spanning tree

A spanning tree of a graph $\langle V, E \rangle$ is a tree $\langle V, E' \rangle$ with $E' \subseteq E$.

The graph  has eight different spanning trees:



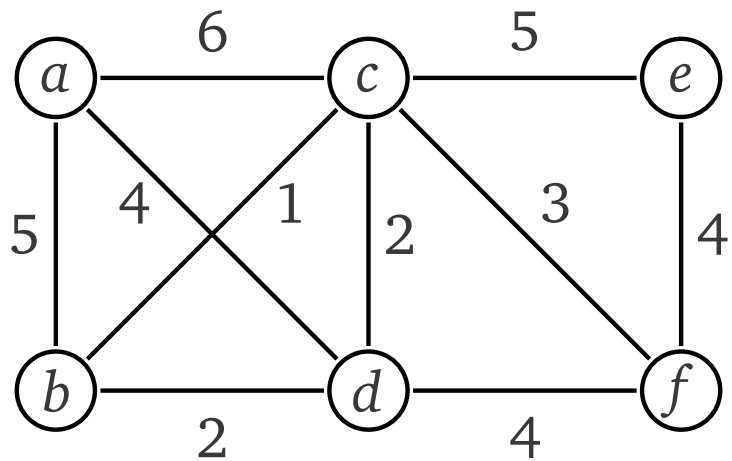
Minimum Spanning Trees of Weighted Graphs

Distances or cost of edges

Means some spanning trees are preferred

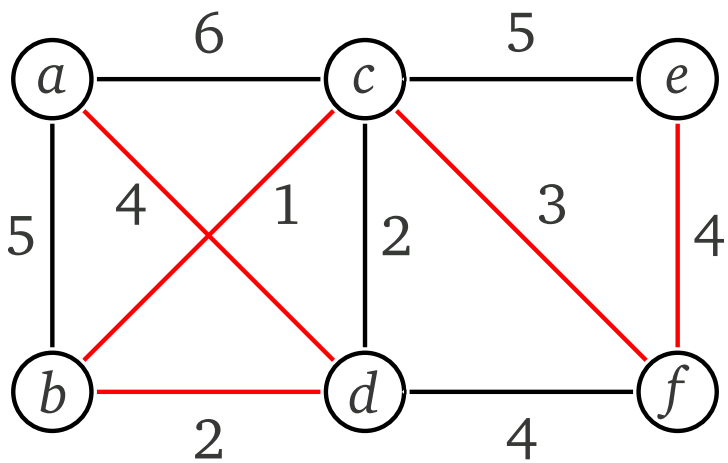
weighted graph problem

Find a spanning tree with the smallest possible cost

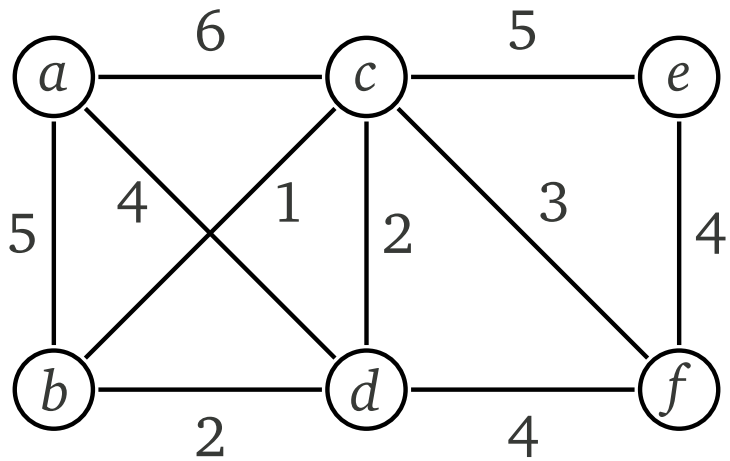


Minimum Spanning Trees

Given a weighted graph, a sub-graph which is a tree with minimal weight is a minimum spanning tree for the graph.



Towards Prim's algorithm



Minimum Spanning Trees: Prim's Algorithm

Greedy Algorithm

- sequence of subtrees T
- each adding a node together with an edge to a node in the previous subtree
- always pick a closest node from outside the tree

Minimum Spanning Trees: Prim's Algorithm

Greedy Algorithm

- sequence of subtrees T
- each adding a node together with an edge to a node in the previous subtree
- always pick a closest node from outside the tree

function PRIM($\langle V, E \rangle$)
  $V_T \leftarrow \{v_0\}$
  $E_T \leftarrow \emptyset$
  for $i \leftarrow 1$ to $|V| - 1$ **do**
    find a minimum-weight edge $(v, u) \in V_T \times (V \setminus V_T)$
    $V_T \leftarrow V_T \cup \{u\}$
    $E_T \leftarrow E_T \cup \{(v, u)\}$
  return E_T

Prim's Algorithm

- In each iteration the tree grows by one edge
- Include the node from outside that has the smallest cost
- But how to find the minimum-weight edge (v, u)?

Prim's Algorithm

- In each iteration the tree grows by one edge
- Include the node from outside that has the smallest cost
- But how to find the minimum-weight edge (v, u)?
- Keep potential nodes in a priority queue
  - organised in a min-heap by edge cost
- Which nodes are connected?

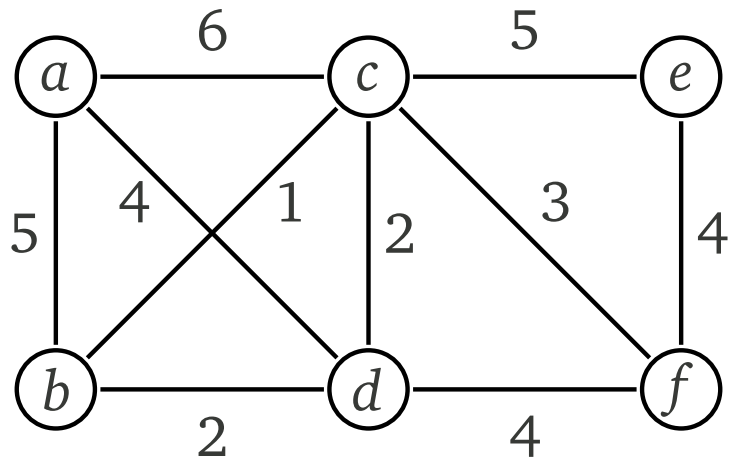
Prim's Algorithm

- In each iteration the tree grows by one edge
- Include the node from outside that has the smallest cost
- But how to find the minimum-weight edge (v, u)?
- Keep potential nodes in a priority queue
  - organised in a min-heap by edge cost
- Which nodes are connected?
- Array *prev* of nodes, indexed by V
  - When (v, u) is included set $prev[u] = v$

Prim's Algorithm

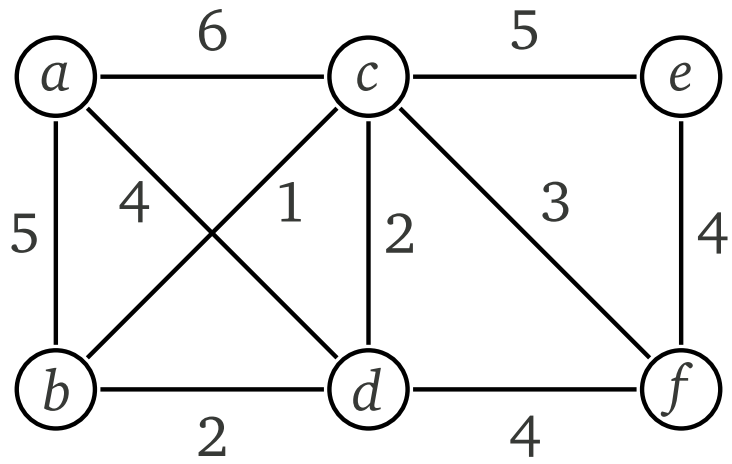
function PRIM($\langle V, E \rangle$)
    for each $v \in V$ **do**
        $cost[v] \leftarrow \infty$
        $prev[v] \leftarrow nil$
    pick initial node v_0
    $cost[v_0] \leftarrow 0$
    $Q \leftarrow$ INITPRIORITYQUEUE(V) ▷ priorities are cost values
    while Q is non-empty **do**
        $u \leftarrow$ EJECTMIN(Q)
        for each $(u, w) \in E$ **do**
            if $weight(u, w) < cost[w]$ **then**
                $cost[w] \leftarrow weight(u, w)$
                $prev[w] \leftarrow u$
                UPDATE($Q, w, cost[w]$) ▷ rearranges priority queue

Prim's Algorithm: Example



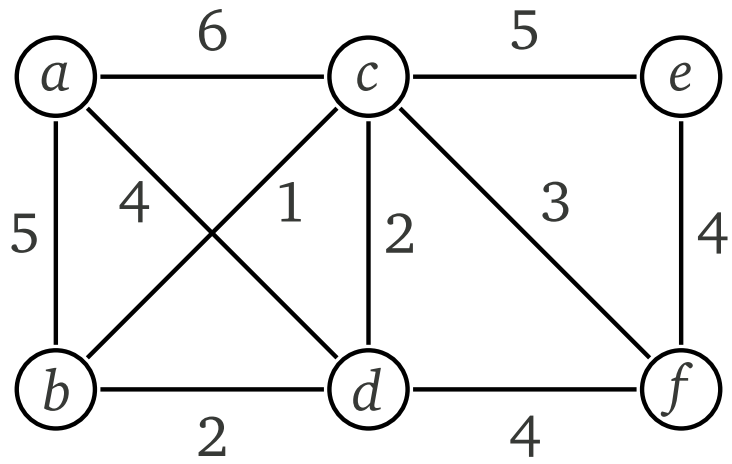
Tree T	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil

Prim's Algorithm: Example



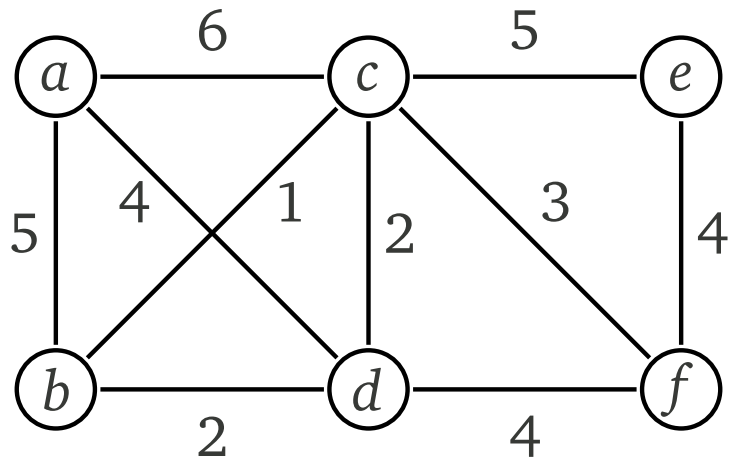
Tree T	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
a		$5/a$	$6/a$	$4/a$	∞/nil	∞/nil

Prim's Algorithm: Example



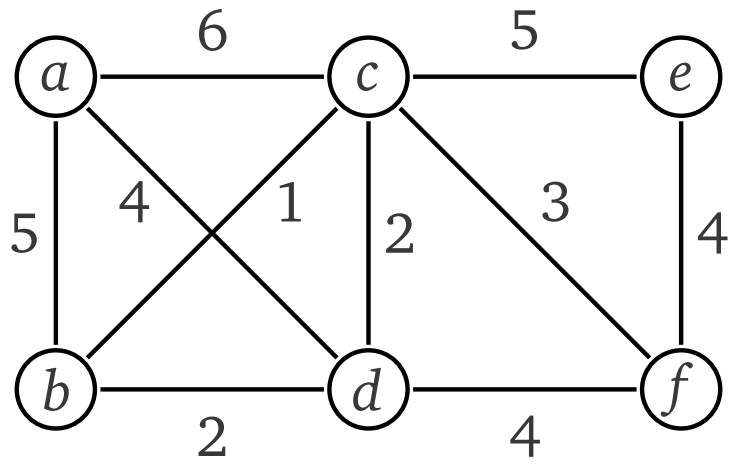
Tree T	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
a		$5/a$	$6/a$	$4/a$	∞/nil	∞/nil
a,d		$2/d$	$2/d$		∞/nil	$4/d$

Prim's Algorithm: Example



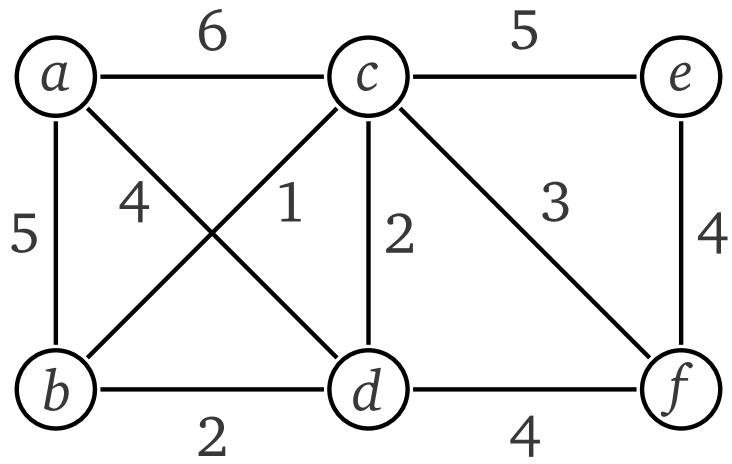
Tree T	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
a		$5/a$	$6/a$	$4/a$	∞/nil	∞/nil
a,d		$2/d$	$2/d$		∞/nil	$4/d$
a,d,b			$1/b$		∞/nil	$4/d$

Prim's Algorithm: Example



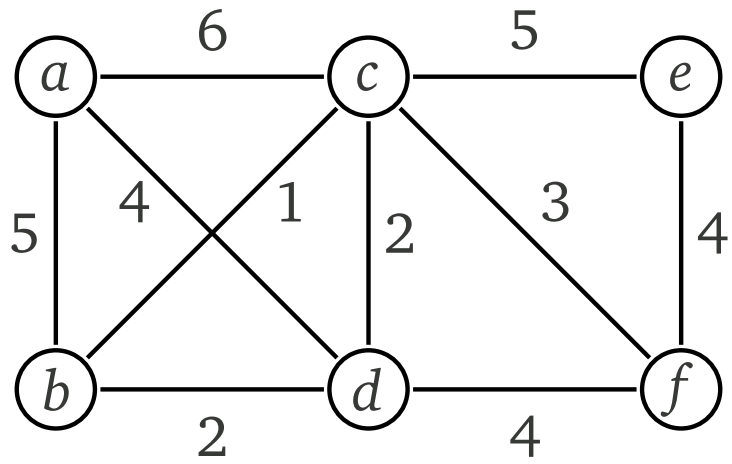
Tree T	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
a		$5/a$	$6/a$	$4/a$	∞/nil	∞/nil
a,d		$2/d$	$2/d$		∞/nil	$4/d$
a,d,b			$1/b$		∞/nil	$4/d$
a,d,b,c					$5/c$	$3/c$

Prim's Algorithm: Example



Tree T	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
a		$5/a$	$6/a$	$4/a$	∞/nil	∞/nil
a,d		$2/d$	$2/d$		∞/nil	$4/d$
a,d,b			$1/b$		∞/nil	$4/d$
a,d,b,c					$5/c$	$3/c$
a,d,b,c,f					$4/f$	

Prim's Algorithm: Example



Tree T	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
a		$5/a$	$6/a$	$4/a$	∞/nil	∞/nil
a,d		$2/d$	$2/d$		∞/nil	$4/d$
a,d,b			$1/b$		∞/nil	$4/d$
a,d,b,c					$5/c$	$3/c$
a,d,b,c,f					$4/f$	
a,d,b,c,f,e						

Analysis of Prim's Algorithm

Naive data structures

- For each node, we look through the edges
- find those incident to the node
- pick the one with smallest cost
- Altogether $O(|V| \cdot |E|)$

Cleverer data structures

- adjacency lists for the graph and a min-heap for the priority queue
- we perform $|V| - 1$ heap deletions (each at cost $O(\log |V|)$)
- $|E|$ updates of priorities (again, each at cost $O(\log |V|)$).
- Altogether $(|V| - 1 + |E|)O(\log |V|)$
- In a connected graph, $|V| - 1 \leq |E|$, so this is $O(|E| \log |V|)$.

Kruskal's Algorithm (Not Examinable)

Kruskal's Algorithm (Not Examinable)

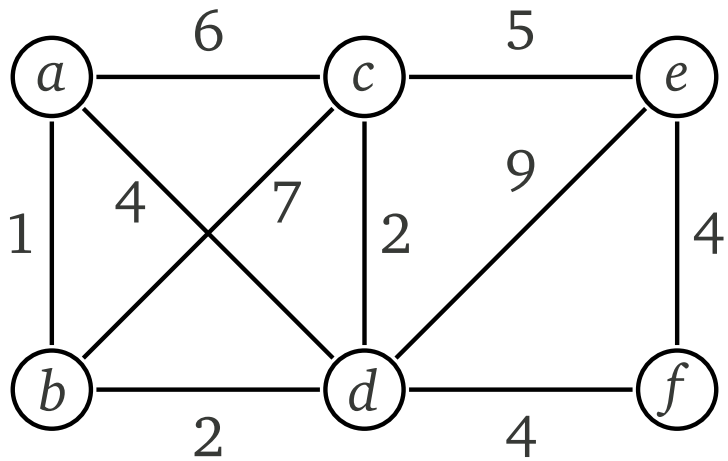
- alternative minimal-spanning-tree algorithm
- also greedy
- explained in Levitin's Section 9.2
- for sparse graphs Kruskal's algorithm better than Prim's.

Dijkstra's Algorithm

- shortest-path algorithm from a fixed start node[1]
- (directed or undirected) weighted graphs
- complexity same as that of Prim's algorithm

[1] Recall Floyd's algorithm: $O(|V|^3)$

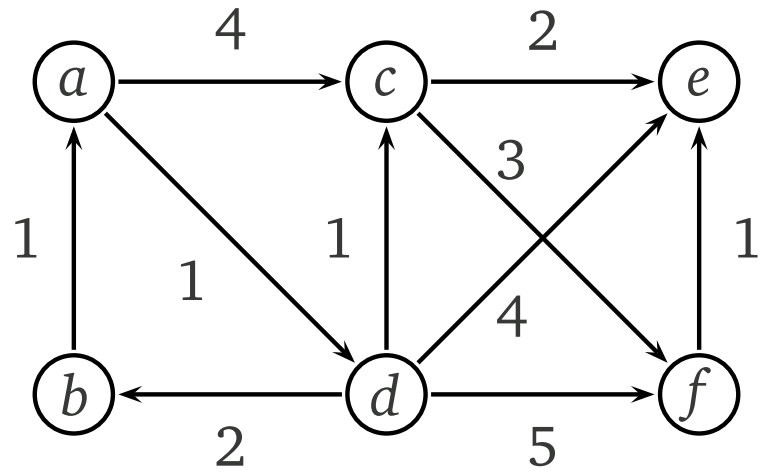
Towards Dijkstra's algorithm



Dijkstra's Algorithm

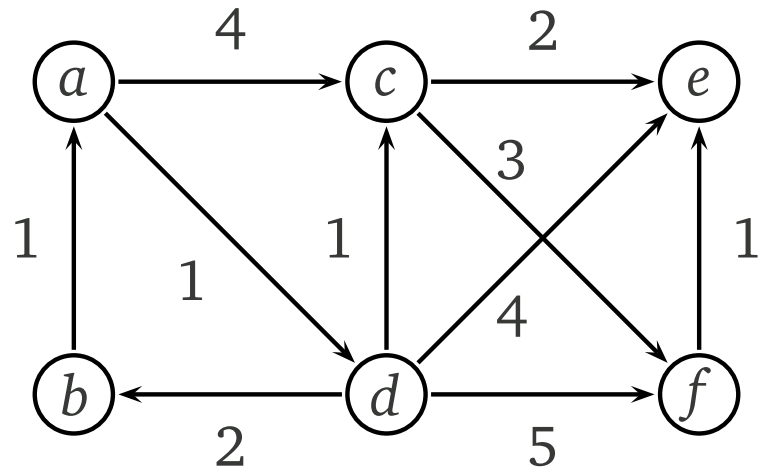
function DIJKSTRA($\langle V, E \rangle, v_0$)
    for each $v \in V$ **do**
        $dist[v] \leftarrow \infty$
        $prev[v] \leftarrow nil$
    $dist[v_0] \leftarrow 0$
    $Q \leftarrow$ INITPRIORITYQUEUE(V) ▷ priorities are distances
    while Q is non-empty **do**
        $u \leftarrow$ EJECTMIN(Q)
        for each $(u, w) \in E$ **do**
            if $dist[u] + weight(u, w) < dist[w]$ **then**
                $dist[w] \leftarrow dist[u] + weight(u, w)$
                $prev[w] \leftarrow u$
                UPDATE($Q, w, dist[w]$) ▷ rearranges priority queue

Dijkstra's Algorithm: Example



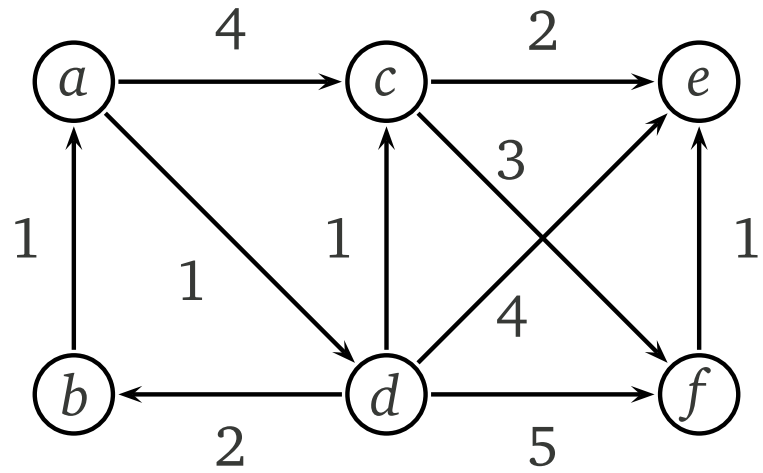
Covered	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil

Dijkstra's Algorithm: Example



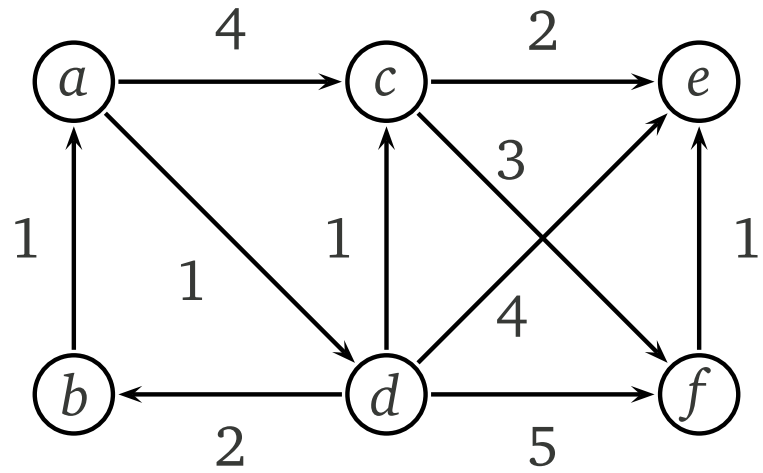
Covered	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
a		∞/nil	$4/a$	$1/a$	∞/nil	∞/nil

Dijkstra's Algorithm: Example



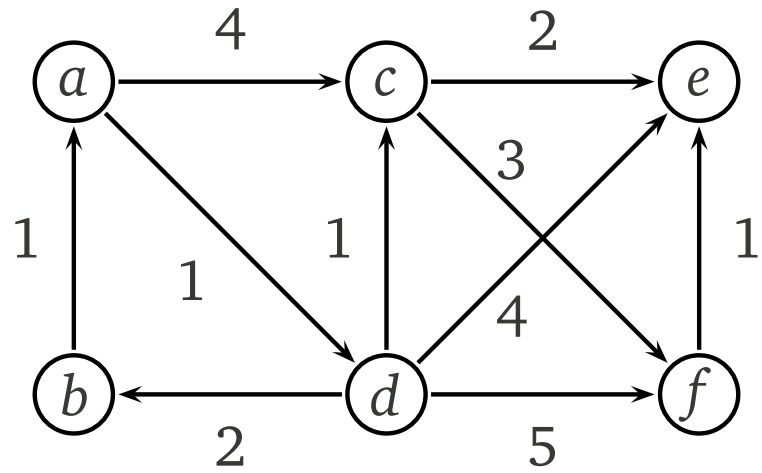
Covered	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
a		∞/nil	$4/a$	$1/a$	∞/nil	∞/nil
a, d		$3/d$	$2/d$		$5/d$	$6/d$

Dijkstra's Algorithm: Example



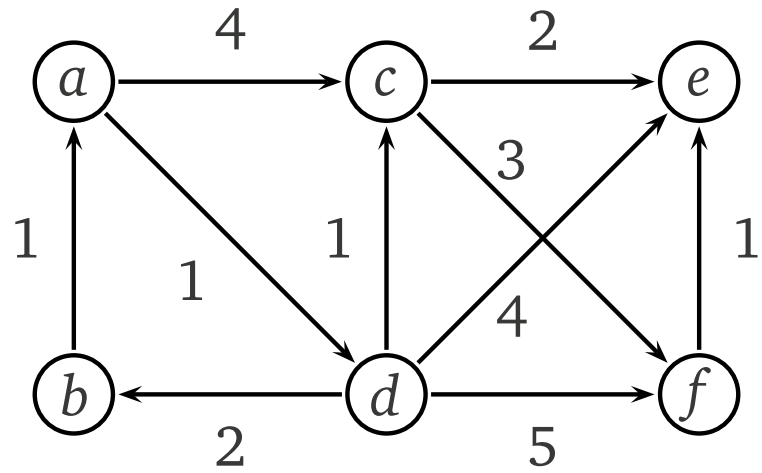
Covered	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
a		∞/nil	$4/a$	$1/a$	∞/nil	∞/nil
a,d		$3/d$	$2/d$		$5/d$	$6/d$
a,d,c		$3/d$			$4/c$	$5/c$

Dijkstra's Algorithm: Example



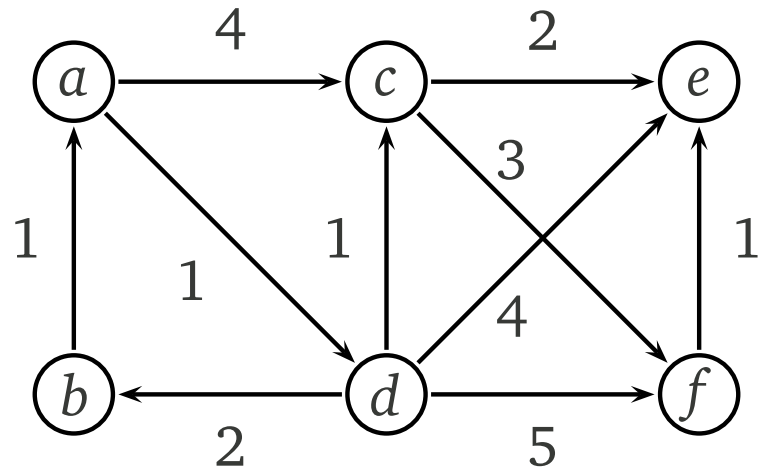
Covered	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
a		∞/nil	$4/a$	$1/a$	∞/nil	∞/nil
a,d		$3/d$	$2/d$		$5/d$	$6/d$
a,d,c		$3/d$			$4/c$	$5/c$
a,d,c,b					$4/c$	$5/c$

Dijkstra's Algorithm: Example



Covered	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
a		∞/nil	$4/a$	$1/a$	∞/nil	∞/nil
a,d		$3/d$	$2/d$		$5/d$	$6/d$
a,d,c		$3/d$			$4/c$	$5/c$
a,d,c,b					$4/c$	$5/c$
a,d,c,b,e						$5/c$

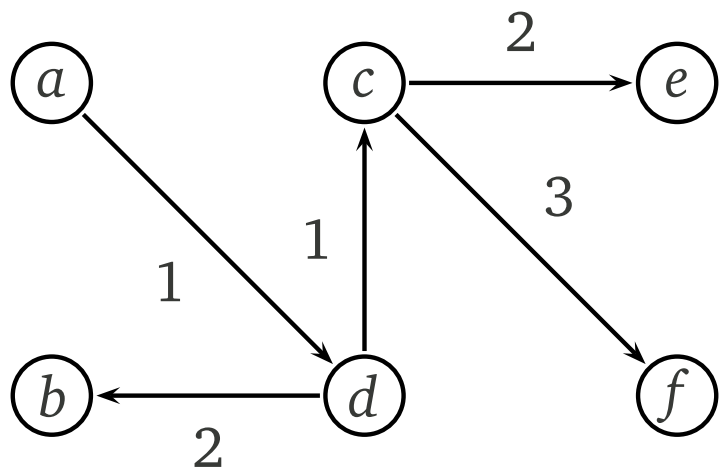
Dijkstra's Algorithm: Example



Covered	a	b	c	d	e	f
—	$0/nil$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
a		∞/nil	$4/a$	$1/a$	∞/nil	∞/nil
a,d		$3/d$	$2/d$		$5/d$	$6/d$
a,d,c		$3/d$			$4/c$	$5/c$
a,d,c,b					$4/c$	$5/c$
a,d,c,b,e						$5/c$
a,d,c,b,e,f						

Dijkstra's Algorithm: Tracing Paths

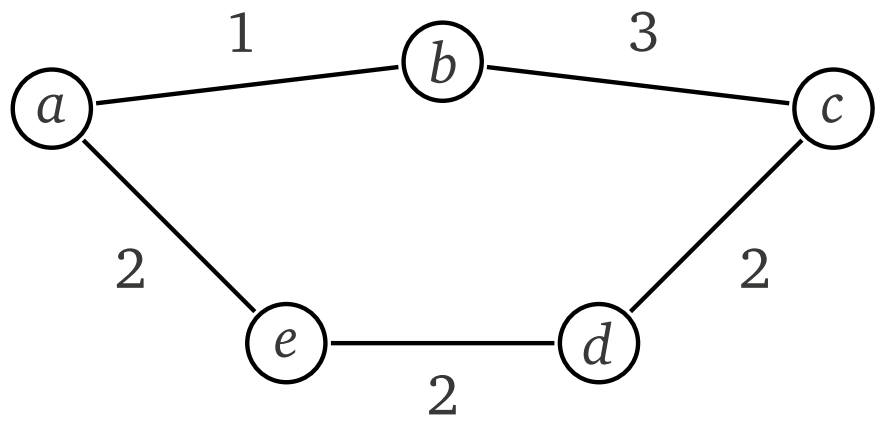
The array `prev` is not really needed, unless we want to retrace the shortest paths from node a:



This tree is referred to as the shortest-path tree.

Spanning Trees and Shortest-Path Trees

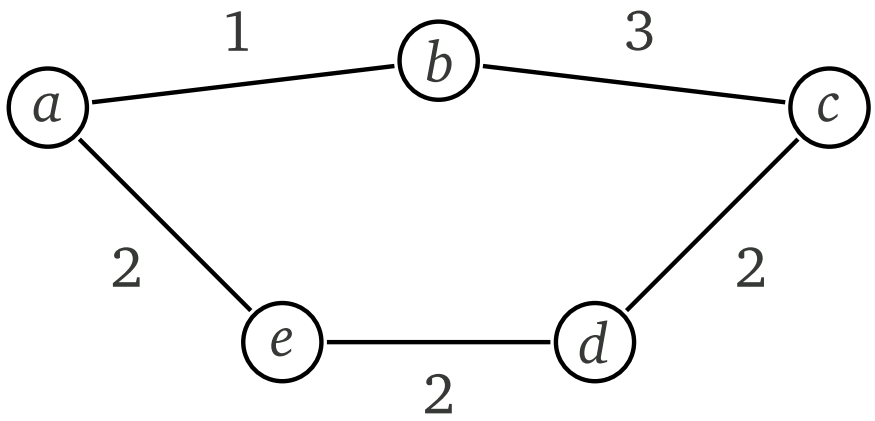
The shortest-path tree that results from Dijkstra's algorithms is very similar to a minimal spanning tree, but it need not be a minimal spanning tree.



Which edge is missing in the minimal spanning tree?

Which edge is missing in the shortest-path tree, assuming we are interested in paths from a?

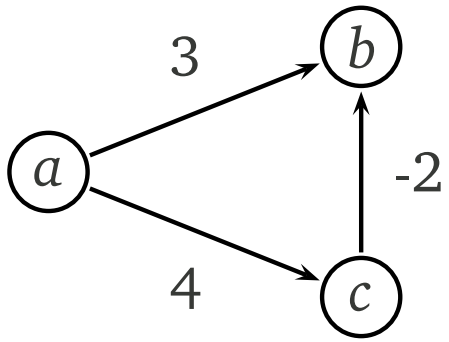
Example



Negative Weights

We focused on positive weights: Dijkstra's algorithm may not work otherwise!

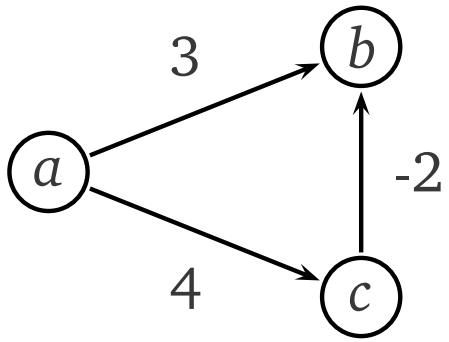
Greedy chooses edge from a to b—is clearly the wrong one.



Negative Weights

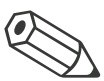
We focused on positive weights: Dijkstra's algorithm may not work otherwise!

Greedy chooses edge from a to b—is clearly the wrong one.



The Bellman-Ford algorithm will find single-source shortest paths in arbitrary weighted graphs (more costly to run).

What if there is a cycle with accumulated negative weight?



Summary

Today

Greedy algorithms for graph problems

Next lecture

Greedy algorithm for data compression: Huffman encoding